

摊牌了！策略模式在项目设计中用得最多

日常 Coding 过程中，设计模式三板斧：模版、构建者、策略，今天来说下第三板斧 **策略设计模式**。

策略模式还是比较简单并且使用较多的，平常我们多运用策略模式用来消除 if-else、switch 等多重判断的代码，消除 if-else、switch 多重判断 **可以有效应对代码的复杂性**。

如果分支判断会不断变化（增、删、改），那么可以使用别的技巧让其满足开闭原则，提高代码的扩展性（策略模式场景主要负责解耦，开闭原则需要额外支持）。

下文中会详细列举如何使用设计模式做个 Demo、模式的真实场景以及策略模式的好处。

什么是策略模式

策略模式在 GoF 的《设计模式》一书中，是这样定义的：

Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.

定义一组算法类，**将每个算法分别封装起来，让它们可以互相替换**。策略模式使这些算法在客户端调用它们的时候能够互不影响地变化，客户端代指使用算法的代码。

看到上面的介绍可能不太明白策略模式具体为何物，这里会从最基本的代码说起，一步一步彻底掌握此模式。下述代码可能大家都能联想出对应的业务，**根据对应的优惠类型，对价格作出相应的优惠**。

```

public class Example {

    public static void main(String[] args) {
        Double discount = discount("1", 20D);
        System.out.println(discount);
    }

    public static Double discount(String type, Double price) {
        if (Objects.equals(type, "1")) { // 优惠 9.5 折
            return price * 0.95;
        } else if (Objects.equals(type, "2")) { // 优惠 9 折
            return price * 0.9;
        } else if (Objects.equals(type, "3")) { // 优惠 8.5 折
            return price * 0.85;
        } else {
            // 无优惠
            return price;
        }
    }
}

```

这段代码是能够满足项目中业务需求的，而且很多已上线生产环境的代码也有这类代码。但是，这一段代码存在存在两个弊端：

1. 代码的复杂性，正常业务代码逻辑肯定会比这个代码块复杂很多，这也就 **导致了 if-else 的分支以及代码数量过多**。这种方式可以通过将代码拆分成独立函数或者拆分成类来解决。
2. 开闭原则，价格优惠肯定会 **随着不同的时期作出不同的改变**，或许新增、删除或修改。如果在一个函数中修改无疑是件恐怖的事情，想想可能多个开发者分别进行开发，杂乱无章的注释，混乱的代码逻辑等情况十有八九会发生。

如何运用策略模式优化上述代码，使程序设计看着简约、可扩展等特性。

1. 简化代码的复杂性，将不同的优惠类型定义为不同的策略算法实现类。
2. 保证开闭原则，增加程序的健壮性以及可扩展性。

策略模式示例

将上述代码块改造为策略设计模式，大致需要三个步骤。

1. 定义抽象策略接口，因为业务使用接口而不是具体的实现类的话，便可以灵活的替换不同的策略；
2. 定义具体策略实现类，实现自抽象策略接口，其内部封装具体的业务实现；

3. 定义策略工厂，封装创建策略实现（算法），对客户端屏蔽具体的创建细节。

```
public interface DiscountStrategy { // 定义策略抽象接口
    Double discount(Double price);
}

public class Discount95Strategy implements DiscountStrategy { // 95折优惠策略类
    @Override
    public Double discount(Double price) { return price * 0.95; }
}

public class Discount9Strategy implements DiscountStrategy { // 9折优惠策略类
    @Override
    public Double discount(Double price) { return price * 0.9; }
}

public class DiscountStrategyFactory {
    private static final Map<String, DiscountStrategy> discountStrategies = new HashMap(); // 策略存储容器

    static {
        discountStrategies.put("1", new Discount95Strategy());
        discountStrategies.put("2", new Discount9Strategy());
    }

    public static DiscountStrategy chooseStrategy(String type) {
        return discountStrategies.get(type);
    }
}
```

目前把抽象策略接口、具体的策略实现类以及策略工厂都已经创建了，现在可以看一下客户端需要如何调用，又是如何对客户端屏蔽具体实现细节的。

```
public class DiscountApplication {

    public static void main(String[] args) {
        // 从抽象策略工厂中选择具体的策略
        DiscountStrategy discountStrategy = DiscountStrategyFactory.chooseStrategy("1");

        // 执行具体策略
        Double discount = discountStrategy.discount(100);

        // 输出策略执行结果
        System.out.println("优惠后金额 :: " + discount);

        /**
         * 输出结果如下:
         *
         * 优惠后金额 :: 9.5
         */
    }
}
```

根据代码块图片得知，具体策略类是从策略工厂中获取，确实是取消了 if-else 设计，**在工厂中使用 Map 存储策略实现**。获取到策略类后执行具体的优惠策略方法就可以获取优惠后的金额。

通过分析大家得知，目前这种设计确实将应用代码的复杂性降低了。**如果新增一个优惠策略，只需要新增一个策略算法实现类即可。**但是，添加一个策略算法实现，**意味着需要改动策略工厂中的代码**，还是不符合开闭原则。

如何完整实现符合开闭原则的策略模式，需要借助 Spring 的帮助，详细案例请继续往下看。

项目中真实的应用场景

最近项目中设计的一个功能用到了策略模式，分为两类角色，笔者负责定义抽象策略接口以及策略工厂，不同的策略算法需要各个业务方去实现，可以联想到上文中的优惠券功能。因为是 Spring 项目，所以都是按照 Spring 的方式进行处理，话不多说，上代码。

```
public interface DiscountStrategy {
    Double discount(Double price); // 定义抽象策略接口
    String mark(); // 定义策略算法标示
}

@Component
public class Discount95Strategy implements DiscountStrategy {
    @Override
    public Double discount(Double price) { return price * 0.95; }

    @Override
    public String mark() { return "1"; }
}

@Component
public class Discount9Strategy implements DiscountStrategy {
    @Override
    public Double discount(Double price) { return price * 0.9; }

    @Override
    public String mark() { return "2"; }
}
```

可以看到，比对上面的示例代码，有两处明显的变化：

1. 抽象策略接口中，新定义了 mark() 接口，此接口用来标示算法的唯一性；
2. 具体策略实现类，使用 @Component 修饰，将对象本身交由 Spring 进行管理。

小贴士：为了阅读方便，mark() 返回直接使用字符串替代，读者朋友在返回标示时最好使用枚举定义。

接下来继续查看抽象策略工厂如何改造，才能满足开闭原则。

```
@Component
public class DiscountStrategyFactory implements InitializingBean {

    @Autowired
    private ApplicationContext applicationContext;

    private final Map<String, DiscountStrategy> discountStrategies = new HashMap();

    public DiscountStrategy chooseStrategy(String type) {
        return discountStrategies.get(type);
    }

    @Override
    public void afterPropertiesSet() throws Exception {
        // 从 IOC 容器中获取 DiscountStrategy 类型的 Bean 对象，而这些 Bean 对象就是具体的策略算法实现类
        Map<String, DiscountStrategy> discountStrategyMap = applicationContext.getBeansOfType(DiscountStrategy.class);

        // 将具体的策略算法标示以及本身添加到我们自定义的 Map 中，方便屏蔽客户端感知调用
        discountStrategyMap.forEach((key, val) -> discountStrategies.put(val.mark(), val));
    }
}
```

和之前责任链模式相同（TODO 添加链接），都是通过 InitializingBean 接口实现中调用 IOC 容器查找对应策略实现，随后将策略实现 mark() 方法返回值作为 key，策略实现本身作为 value 添加到 Map 容器中等待客户端的调用。

```
@SpringBootTest
class StrategyApplicationTests {

    @Autowired
    private DiscountStrategyFactory discountStrategyFactory;

    @Test
    void contextLoads() {
        // 通过优惠策略工厂获取类型为 1 的具体策略算法
        DiscountStrategy discountStrategy = discountStrategyFactory.chooseStrategy("1");

        // 根据策略算法获取优惠后的价格
        Double discount = discountStrategy.discount(100);

        System.out.println(discount);
    }
}
```

这里使用的 SpringBoot 测试类，注入策略工厂 Bean，通过策略工厂选择出具体的策略算法类，继而通过算法获取到优惠后的价格。小插曲：如果不想把策略工厂注入 Spring 也可以，实现方法有很多。

总结下本小节，我们通过和 Spring 结合的方式，通过策略设计模式对文初的代码块进行了两块优化：应对代码的复杂性，让其满足开闭原则。更具体一些呢就是 **通过抽象策略算法类减少代码**

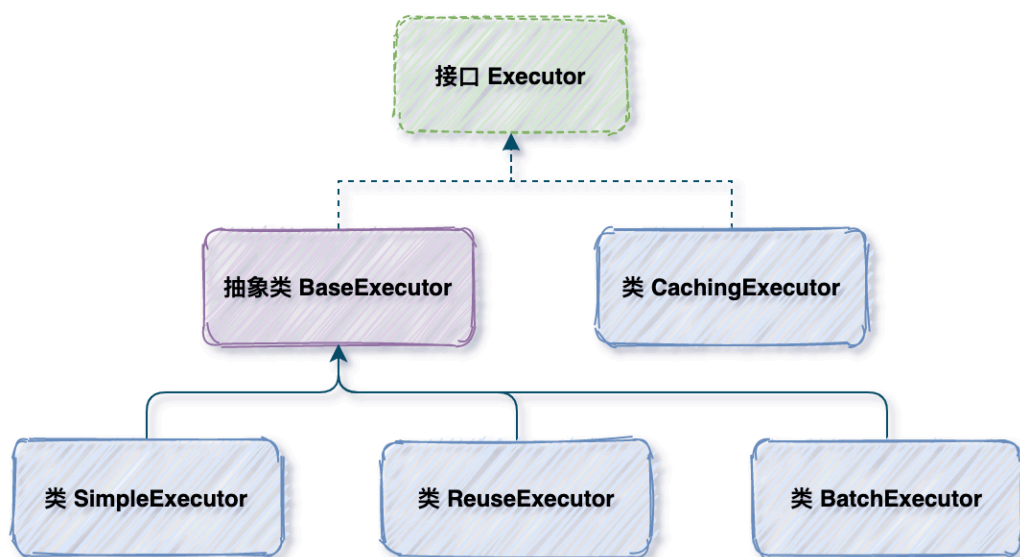
的复杂性，继而通过 Spring 的一些特性同时满足了开闭原则，现在来了新需求只要添加新的策略类即可，健壮易扩展。

源码底层如何耍策略模式

自己用肯定觉得不够，必要时候还得看看设计开源框架源码的大佬们如何在代码中运用策略模式的。

在作者了解中，JDK、Spring、SpringMvc、Mybatis、Dubbo 等等都运用了策略设计模式，这里就以 Mybatis 举例说明。

Mybatis 中 Executor 代表执行器，负责增删改查的具体操作。其中用到了两种设计模式，模版方法以及策略模式。



- Executor 代表了抽象策略接口，刚才说到的模版方法模式源自 BaseExecutor。
- Configuration 代表策略工厂，负责创建具体的策略算法实现类。
- SimpleExecuto、ReuseExecutor... 表示封装了具体的策略算法实现类。

```

public Executor newExecutor(Transaction transaction, ExecutorType executorType) {
    executorType = executorType == null ? defaultExecutorType : executorType;
    executorType = executorType == null ? ExecutorType.SIMPLE : executorType;

    Executor executor;
    if (ExecutorType.BATCH == executorType) {
        executor = new BatchExecutor(this, transaction);
    } else if (ExecutorType.REUSE == executorType) {
        executor = new ReuseExecutor(this, transaction);
    } else {
        executor = new SimpleExecutor(this, transaction);
    }

    if (cacheEnabled) {
        executor = new CachingExecutor(executor);
    }

    executor = (Executor) interceptorChain.pluginAll(executor);
    return executor;
}

```

上述代码块发生在 Configuration 类中创建执行器 Executor，通过 executorType 判断创建不同的策略算法。

上述代码块并没有彻底消除 if-else，因为 Mybatis 中执行器策略基本是固定的，也就是说它只会有这些 if-else 判断，基本不会新增或修改。如果非要消除 if-else，可以这么搞，这里写一下伪代码。

```

static final Map<ExecutorType, Executor> executors = new HashMap();

static {
    executors.put(ExecutorType.SIMPLE, new SimpleExecutor(this, transaction));
    executors.put(ExecutorType.BATCH, new BatchExecutor(this, transaction));
    ...
}

public Executor newExecutor(Transaction transaction, ExecutorType executorType) {
    executorType = executorType == null ? defaultExecutorType : executorType;
    executorType = executorType == null ? ExecutorType.SIMPLE : executorType;

    return executors.get(executorType);
}

```

这种方式叫做“查表法”，通过策略工厂实现消除 if-else 分支。最后，Mybatis 太过详细的设计这里不再赘述，有兴趣的小伙伴可以去把源码下载啃一啃。

到了这里可能有读者看出了问题，策略模式就算消除了 if-else 但是如果添加新的策略类，不还是会违反开闭原则么？

没错，因为 Mybatis 本身没有引入 Spring 依赖，**所以没有办法借助 IOC 容器实现开闭原则**。Spring 是一种开闭原则解决方式，那还有没有别的解决方式？

解决方式有很多，开闭原则核心就是 **对原有代码修改关闭，对新增代码开放**。可以通过扫描指定包下的自定义注解亦或者通过 instanceof 判断是否继承自某接口都可以。不过，项目如果用了 Spring 还是消停的吧。

文末结语

文章中图文并茂的方式介绍策略模式，通过价格优惠的场景，进而引用本文的重点：策略设计模式，相信小伙伴看完后都会有一定的收获。

策略模式的本质依然是对代码设计解耦合，通过三类角色贯穿整个策略模式：抽象策略接口、策略工厂以及具体的策略实现类。通过细粒度的策略实现类避免了主体代码量过多，减少了设计中的复杂性。

作者听到过很多小伙伴觉得自己做的都是 CRUD 工作，没有挑战性没意思。其实，我想说的是：**业务代码一样牛逼，一样能体现程序员的水平**。不一定非要高并发、大数据等场景。颇有一屋不扫何以扫天下的意思。

最后抛出一个问题：**出现 if-else 的代码，一定要使用策略模式优化么？**

如果 if-else 判断分支不多并且是固定的，后续不会出现新的分支，那我们完全 **可以通过抽函数的方式降低程序复杂性**；不要想法设法去除 if-else 语句，存在即合理。而且，使用策略模式会导致类增多，没有必要为了少量的判断分支引入策略模式。

参考文章：

- 《设计模式之美：策略模式》

更新: 2023-07-17 15:34:11

原文: <<https://www.yuque.com/magestack/12306/wnld8znw3ul4a2l8>>